



 Latest updates: <https://dl.acm.org/doi/10.1145/3725325>

RESEARCH-ARTICLE

MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search

SAIRAJ VORUGANTI, University of Waterloo, Waterloo, ON, Canada

M. TAMER ÖZSU, University of Waterloo, Waterloo, ON, Canada

Open Access Support provided by:

University of Waterloo



Published: 18 June 2025

[Citation in BibTeX format](#)



MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search

SAIRAJ VORUGANTI, University of Waterloo, Canada

M. TAMER ÖZSU, University of Waterloo, Canada

Approximate nearest neighbor search (ANNS) on high dimensional vectors is important for numerous applications, such as search engines, recommendation systems, and more recently, large language models (LLMs), where Retrieval Augmented Generation (RAG) is used to add context to an LLM query. Graph-based indexes built on these vectors have been shown to perform best but have challenges. These indexes can either employ refinement-based construction strategies such as K-Graph and NSG, or increment-based strategies such as HNSW. Refinement-based approaches have fast construction times, but worse search performance and do not allow for incremental inserts, requiring a full reconstruction each time new vectors are added to the index. Increment-based approaches have good search performance and allow for incremental inserts, but suffer from slow construction. This work presents MIRAGE-ANNS (Mixed Incremental Refinement Approach Graph-based Exploration for Approximate Nearest Neighbor Search) that constructs the index as fast as refinement-based approaches while retaining search performance comparable or better than increment-based ones. It also allows incremental inserts. We show that MIRAGE achieves state of the art construction and query performance, outperforming existing methods by up to $2\times$ query throughput on real-world datasets.

CCS Concepts: • **Information systems** → **Information retrieval**.

Additional Key Words and Phrases: Vector Search, Graph-based index, Nearest neighbor search

ACM Reference Format:

Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 188 (June 2025), 27 pages. <https://doi.org/10.1145/3725325>

1 Introduction

Advancements in deep learning have allowed for multiple modalities of data, such as images and text, to be leveraged. The multi-modal data is mapped into dense vectors in a vector space [5], and the models are trained in such a way that the “distance” between two vectors represents the similarity between the objects themselves. With the growing popularity of vector embeddings, there has been a surge in vector database offerings, including Milvus [51], Pinecone [7], and Vespa [8]. Traditional database management systems (DBMS) have also incorporated vector data [6]. Similarity search over this vector space is a fundamental operation that allows for more precise and contextualized search over multi-modal data [32]. The search finds the k nearest neighbors of a given point based on the vector distance. Systems integrate vector similarity search into query processing to support workloads that require finding the most similar vectors to a query vector. More recently, similarity search is used in systems supporting vector data in order to perform Retrieval Augmented Generation (RAG), helping to add context to an LLM.

Authors' Contact Information: Sairaj Voruganti, sairajv@uwaterloo.ca, University of Waterloo, Waterloo, Ontario, Canada; M. Tamer Özsu, tamer.ozsu@uwaterloo.ca, University of Waterloo, Waterloo, Ontario, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/6-ART188

<https://doi.org/10.1145/3725325>

The *k*-Nearest Neighbor Search (kNNS) problem is computationally hard to solve in high-dimensional spaces due to the curse of dimensionality [24]. In the worst-case, this means that once the number of dimensions becomes too high, an expensive linear scan is required. Most real-world use cases can afford approximate solutions, leading to the definition of *approximate nearest neighbor search* (ANNS) problem. ANNS typically relies on an index over the vector dataset. Indexes can be roughly categorized into four types: hashing-based [19], tree-based [11, 12], graph-based [30, 45], and quantization-based [31]. Recent studies [53] have empirically shown that graph-based indexes perform the best, achieving the highest recall and throughput on a variety of datasets.

Graph-based indexes construct a *proximity graph* on the data set, where the vertices represent the vectors and the edges represent a neighbor relationship between them. The main types of approaches for building graph-based indexes are refinement-based and increment-based [34]. Refinement-based approaches start with a random initial graph and then connect each vertex to its *k*-nearest neighbors, improving the quality of the graph. Examples of this approach include KGraph [21], NSG [16], NN-Descent [21], and RNNDescent [34]. The search performance of refinement-based approaches in terms of recall and QPS (Queries-per-Second) can be low without additional processing post-construction, and they do not allow for incremental inserts, meaning that inserting a new vector requires a full index reconstruction.

Increment-based approaches, on the other hand, traditionally start with an empty graph and add points one at-a-time. Each new insertion is treated as a query point and a nearest neighbor search is performed on the current graph to find its *k* nearest neighbors for edge connections. The most popular example of an increment-based approach is HNSW [30], which is used by many commercial offerings due to its superior search performance. HNSW constructs a hierarchical graph where the bottom layer (Layer 0) contains all the points in the dataset, while each higher layer contains increasingly fewer points. The increment-based approaches have good search performance [9] and allow incremental insertions, but have a quite high index construction time due to performing nearest neighbor search on the whole graph for every point being added.

Thus, there is a trade-off between refinement-based indexes with fast index construction times but inferior search quality and increment-based indexes with slow index construction times and superior search quality [36]. For example, in our experiments on the Tiny5m [1] dataset, RNNDescent can construct its graph around 8× faster than HNSW; however, it achieves around 50% of HNSW QPS during search (*k* = 100).

In this paper, we introduce MIRAGE, a new graph-based index that tackles the tradeoff between index construction time and query performance: it has a fast index construction time as well as high search performance.

This paper makes the following contributions:

- We propose MIRAGE, a new approach to constructing graph-based indexes using refinement-based construction to create the bottom layer of the graph, and then building layers above in an increment-based manner, with upper layers containing fewer points and longer links, and lower layers containing more points and shorter links. The search uses a greedy algorithm starting from the highest layer and continuing down to the lowest layer. MIRAGE allows for incremental inserts without a full graph reconstruction. It is built on top of the FAISS library [14], a popular open-source library for indexing and performing ANNS.
- We evaluate MIRAGE on real-world datasets of different difficulties as measured by local intrinsic dimensionality (LID) [25], showing that it constructs the index as much as 6× faster than HNSW and 7× faster than NSG, current state of the art approaches, while also improving search efficiency by as much as 2× over the said approaches, where efficiency is measured by QPS (Queries-per-Second) at a fixed recall.

- Through ablation studies and studies on the properties of MIRAGE and other graph-based indexes, we highlight the properties that impact graph-based index performance, and how they can be combined in novel ways to achieve superior performance.

The rest of the paper is structured as follows: Section 2 introduces basic definitions and background information on graph-based indexes. In Section 3 we introduce MIRAGE. In Section 4 we present our experiments comparing MIRAGE to existing methods and our ablation studies. Section 5 discusses related work. Finally, we conclude in Section 6 and discuss future directions.

2 Background

2.1 Basic Definitions

Definition 1. (Vector dataset) A vector dataset $P = \{p_0, p_1, \dots, p_{n-1}\}$ has each element p_i as a vector $\mathbf{x}^i = [x_0^i, x_1^i, \dots, x_{d-1}^i]$ of d dimensions. When clear from the context, the superscript is omitted for clarity.

Definition 2. (Distance) The distance between two vectors $\mathbf{x}, \mathbf{y}$ in P , $\delta(\mathbf{x}, \mathbf{y})$ can be computed by several distance functions. The most commonly used distance function is the Euclidean distance (also called l_2 norm).

Definition 3. (kNNS and kANNS) Given a vector set P and a query point q , k -nearest neighbor search ($kNNS(P, q, k)$) identifies a set of k vectors ($K \subseteq P$) with the smallest distance to q , i.e. $\max_{p \in K} |p, q| \leq \min_{p \in P \setminus K} |p, q|$. The k -approx NNS ($kANNS(P, q, k)$) is a relaxation of this problem such that vectors in K approximately satisfy this constraint.

Definition 4. (Recall) Given the vector set P and a query point q , let K be the result of $kNNS(P, q, k)$, and K' be a result $kANNS(P, q, k)$ produced by a particular kANNS algorithm A . The $k@k'$ recall of A for q is then defined as $\frac{|K \cap K'|}{|K'|}$ where $k = |K|$ and $k' = |K'|$.

Definition 5. (Average Distance Ratio) Given a set of data vectors P and a query point q , let $d(q, p)$ denote the true distance between q and a data point $p \in P$, and $\hat{d}(q, p)$ be the approximate distance produced by a given algorithm. The average distance ratio is defined as:

$$ADR(q) = \frac{1}{|P|} \sum_{p \in P} \frac{\hat{d}(q, p)}{d(q, p)},$$

where $|P|$ is the number of data points in P . The Average Distance Ratio evaluates the relative error of the algorithm's distance estimations over the dataset.

The notation used in this paper is summarized in Table 1.

2.2 Vector Indexes

Indexes for vector datasets can be categorized into four classes: graph-based, hashing-based, tree-based, and quantization-based. In this paper we focus on graph-based indexes, which are covered in detail in the upcoming sections.

Hashing methods are a family of algorithms that partition the data space into buckets, such that similar items are more likely to be in the same bucket. One of the most popular techniques in this category is Locality-Sensitive Hashing (LSH) [19]. LSH drastically reduces the number of distance comparisons, leading to faster query times.

Tree-based methods, such as SPANN [11, 12] and ANNOY [2] partition the data space into nested regions and use hierarchical data structures to represent nested partitioning, facilitating efficient search. During the search, the algorithm traverses the tree from the root to the leaf nodes, checking

Table 1. Summary of Notation

Symbol	Description
P	Vector dataset consisting of p points
$\mathbf{x}^i$	Vector corresponding to point $p_i \in P$
d	Dimension of each vector $\mathbf{x}$
δ	Distance between two vectors
q	Query point
C	Candidate vertices in graph index – priority queue
K	Closest N points so far to query point q
S	Out-degree limit of the initial random graph
M	Degree bound for vertices in MIRAGE
$iter$	Number of graph refinements in Layer 0 of MIRAGE
R	Number of times reverse edges are added in Layer 0 of MIRAGE
$m_L = 1/\ln M$	Level normalization constant for index
n	Size of dataset

the distance to the query point and backtracking as necessary to explore other branches. Tree-based methods are powerful in searching for the nearest neighbor in low-dimensional space, but they suffer from the curse of dimensionality.

Quantization-based methods such as product quantization [31] compress the high-dimensional vectors into short codes to reduce the space overhead in ANN search. This lowers the memory usage of the index and can speed up the search process. The downside is that recall can suffer due to searching on quantized vectors.

2.3 Graph-based Indexes and ANNS

A graph-based index builds a directed graph $G = (V, E)$ as the index, where each vertex $v_i \in V$ represents a vector in the dataset and each directed edge $e_j \in E$ represents the proximity of two vectors. Let (u, v) be a pair of vertices in the graph corresponding to two vectors and let $\delta(u, v)$ be the distance between them. The edge connecting u and v is considered short or long if $\delta(u, v)$ is small or large, respectively [22]. This is called the *proximity graph*.

ANNS using graph-based indexes typically performs a greedy beam search [54] over the proximity graph in order to find the nearest neighbors of a query (Algorithm 1). Depending on the method, the search begins either at a random or chosen entry point and iteratively moves closer to the target. During this procedure, two priority queues are used, C and K , which are sorted according to proximity to the query. The queue C holds the points scheduled for examination, while K keeps only the closest k points that have already been evaluated. At each iteration, the algorithm removes the foremost point from C to inspect all its neighbors. Should the distance from any of these neighboring points to the query be less than that of the farthest point in K , that neighbor is added to both C and K . The process terminates once the nearest point in C is further away than any point in K . Ultimately, the algorithm outputs the elements found in K .

A skewed dataset distribution leads to the presence of hubs (vertices with high out-degrees) within the graph. These hubs have a high visit frequency and are more likely to link to other hubs [54], which can impact the navigability of the graph, increasing the likelihood of the greedy search to get caught in a local optimum. To ensure effective navigability, hubs require a high degree and a greater number of long-range connections. A graph-based index should ideally contain a mix of long edges, which allow speedy navigation from the starting point towards the region close to the query point, and short edges, which allow the search to converge quickly in the target region and improve accuracy. One way of doing this is to prune certain edges. The RNG Strategy [50] is a

Algorithm 1 Search on Proximity Graph [58]

Require: A proximity graph $G = (V, E)$, a query point q , and the number of k nearest neighbors to return
Ensure: A set K of k nearest neighbors of q in V

```

1:  $C := \emptyset$ ;                                ▶ Candidates sorted closest to furthest from the query
2:  $K := \emptyset$ ;                                ▶ top  $k$  result so far sorted furthest to closest from  $q$ 
3:  $V := \emptyset$ ;                                ▶ Hashmap storing the visited points
4: for a start vertex  $v_s$ ;
5:  $C := C \cup \{v_s\}; V := V \cup \{v_s\}$ ;
6: while  $|C| > 0$  do
7:    $v_c := C.Min(); C.Pop();$                                 ▶ pop the point  $v_c$ 
8:    $v_f := K.Max();$                                 ▶  $k$ -th closest neighbor so far
9:   if  $\delta(v_c, q) > \delta(v_f, q)$  and  $K.size() = k$  then                                ▶ Ending condition
10:    break;
11:   end if
12:   if  $K.size() > k$  then
13:     $K.Pop();$ 
14:   end if
15:    $K := K \cup \{v_c\}$ ;
16:   for each outgoing neighbor  $u$  of  $v_c$  in  $G$  do
17:    if  $u \notin H$  then
18:       $C := C \cup \{u\}; V := V \cup \{u\}$ ;
19:    end if
20:   end for
21: end while
22: return  $K$ ;

```

method for selecting edges of a graph for ANNS. Given a pair of vertices $(u, v) \in P$ and the distance function $\delta(u, v)$, the RNG is a graph constructed as follows: an edge from u to v is included in the RNG if and only if $\forall u' \in P$, u' cannot prune the edge (u, v) . In particular, u' can prune (u, v) if and only if u' is closer to u than v and also closer to v than u , i.e., $\delta(u, u') < \delta(u, v)$ and $\delta(v, u') < \delta(u, v)$. In other words, the RNG rule removes the longest edge in a triangle in the graph.

In the next sections we will discuss different strategies that graph-based indexes use to construct a graph appropriate for ANNS.

2.4 Refinement-based Graph Construction

Refinement-based graph construction starts with an initial graph and then undergoes a refinement process to improve the quality of the graph (Figure 1). The initial graph is constructed with each vertex randomly connected to k neighbors. Index construction is fast primarily because the randomly-connected graph is available at the start and is refined iteratively to improve its quality, rather than building the graph incrementally. Instead of performing global searches for connections, these methods refine the graph iteratively by focusing on the immediate neighborhoods of vertices [34, 53]. Consequently, the number of distance computations while building the index are moderated.

The simplest refinement-based graph structure is K-Graph [21], where each vertex out-links its k nearest neighbors in the graph. K-graph suffers from poor query performance due to a loss of navigability caused by too high of an out-link density [54], leading to a search getting caught in a local optima/hubs before reaching the global optima. This issue is a common theme in graph-based indexes. While local connections are important for the final recall stage of a search, too many local connections lead to a higher chance of getting stuck in local optima and lowering search accuracy.

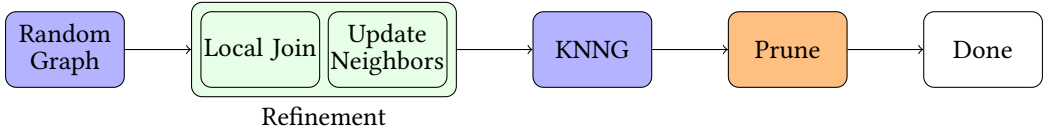


Fig. 1. Refinement-based construction

Since constructing an exact K-Graph is unrealistically time consuming, most solutions turn to NN-Descent [21] to construct an approximate KNN graph. NN-Descent is an iterative, heuristic method for building an approximate k -nearest neighbor graph (k NNG). While a brute-force method has a computational cost of $O(n^2)$ where n is the number of vertices in the dataset, NN-Descent typically operates with a lower empirical cost of $O(n^{1.14})$ [10]. The main idea is the assumption that “the neighbor of my neighbor is likely to be my neighbor.” If vertices v_0 and v_1 are not yet neighbors but are both in the nearest neighbor list of a third vertex v_2 , v_0 and v_1 are likely to be near each other. As a result, v_2 suggests that v_0 and v_1 compare their distances and update their nearest neighbor lists if necessary. NN-Descent begins by randomly initializing a k -NNG and continues to check neighbors until the number of newly identified close neighbors drops below a specified threshold.

There are many algorithms that use NN-Descent in their approaches to build an approximate KNN graph and then use edge-selection strategies to improve performance. One popular such index is NSG [16], which first builds a graph using NN-Descent and then prunes using an edge selection strategy based on the RNG rule. Other examples include Efanna [15], which first builds multiple KD-trees [44] and then executes NN-Descent, and Cagra [35], which builds a graph using NN-Descent on a GPU and then prunes edges based on an edge-selection strategy.

A recent construction strategy called Relative NN-Descent (RNNDescend) [34] has been shown to construct graphs much faster than existing methods, however it can have lower search performance [42]. RNNDescend combines NN-Descent and the RNG rule to speed up the graph construction. It prunes during the initial graph construction process, as opposed to other strategies that postpone pruning until after the creation of the KNN graph. This allows for the graph to be constructed much faster. All the solutions mentioned above add bi-directional/reverse edges to the graph in order to enhance the graph navigability and reduce the number of strong connected components.

2.5 Increment-based Graph Construction

Increment-based graph construction traditionally starts with an empty graph and treats every point being added as a query, performing ANNS on the current graph to find the k nearest neighbors of the newly inserted point (Figure 2). The earliest increment-based construction algorithm is NSW (Navigable Small Worlds) [29], which introduces long-range connections to break up hubs, enhancing the connectivity of the graph. The way NSW forms these connections is with randomization. When inserting a vertex (i.e., a vector in P), NSW performs the greedy search (Algorithm 1) to find neighbors and adds reverse edges from those neighbors. Longer edges will most likely be constructed in the early phases of construction and later become links between the hubs that keep the overall graph connected and allow the logarithmic scaling of the number of hops during greedy routing. Search over NSW can still get caught in a local optima, because the maximum out-degree for each vertex of the graph is not fixed.

HNSW (Hierarchical Navigable Small Worlds) [30], addresses this issue by building a hierarchical structure: building NSW graphs at multiple layers. The bottom layer stores all the vertices in the graph and the number of vertices decreases as higher layers are constructed. The number of points

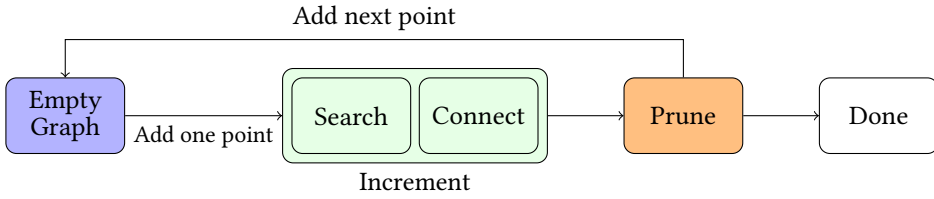


Fig. 2. Increment-based construction

in each layer is reduced exponentially as one moves up, keeping the top layers for long edges and speed, and the bottom layers for shorter edges and accuracy. The idea is based on the skip-list [41], a hierarchical sorted linked list structure that allows for searching and inserting in $O(\log n)$ time complexity on average.

Vertices in HNSW are inserted one-by-one. Every vertex is assigned an integer l , following an exponential decay probability distribution that indicates the maximum layer in which the vertex can be located. For example if $l = 2$, the vertex can be in layers 2, 1 and 0. Once a vertex is assigned an l value, the algorithm starts from the highest assigned layer and performs a nearest neighbor search using the same greedy search strategy mentioned above, looking at C candidates, which is a parameter set by the user, and represents the size of the candidate set C from Algorithm 1. Once C neighbors are found, M (a user-set variable) neighbors are connected to the vertex and the algorithm moves to the next lower layer and continues the process. The process ends when the same search is conducted at the bottom layer (Layer 0).

During search, HNSW starts from the highest layer and performs the greedy beam search until the local nearest neighbor to the query point is found (say w at that layer). Once that happens, the algorithm moves down to the next layer and repeats the same process starting from the vertex that corresponds to w . This is repeated down to the lowest layer where k nearest neighbors are found.

HNSW is the most popular graph-based ANNS algorithm due to its superior search performance, both in query efficiency and accuracy, on hard datasets (i.e., datasets whose local intrinsic dimensionality (LID) is high) [53]. It also allows for incremental inserts, as a new point can be added at any time in the algorithm, without requiring any sort of rebuilding.

Increment-based approaches provide high graph-quality and search performance due to enhanced connectivity [9, 53], while also allowing for incremental inserts, however they have long index build times. Refinement-based approaches are quicker to build, but require a full graph reconstruction for any new inserts and can suffer in search quality on hard datasets if they do not perform any post-processing after refinement. In the next section we introduce MIRAGE, which combines the best of both approaches to construct indexes fast and deliver high quality searches.

3 MIRAGE Index

We now describe MIRAGE, a new graph-based index that provides the fast construction of refinement-based approaches and the superior search performance and incremental insert capabilities of increment-based approaches.

3.1 Motivation and Intuition

The intuition behind MIRAGE lies in understanding the keys to a high-performing graph-based index: efficient construction, global navigability, local precision. As mentioned in Section 2, graphs work well for ANNS when they allow a search to quickly traverse distant regions of the graph to locate the general vicinity of a query point, and then refine the search with precision within

that local neighborhood. However, this balance needs to be achieved while maintaining fast index construction.

In order to ensure efficient construction, MIRAGE indexes all points starting with a randomly connected graph and progressively refines it while pruning at the same time (Figure 3). It follows the refinement-based approach which allows for fast construction times due to the reasons mentioned in Section 2.4.

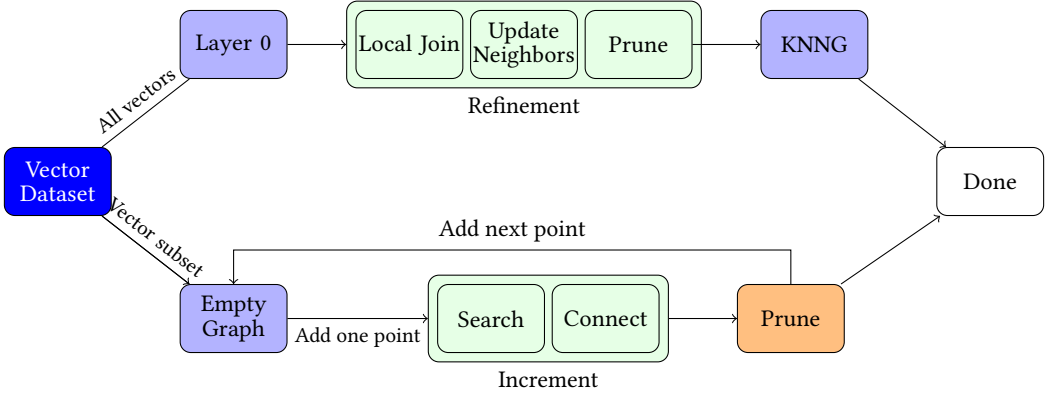


Fig. 3. MIRAGE construction

The challenge for solutions following refinement is that the search over the graph could potentially get caught in local optima reducing their search performance (quality and queries-per-second) unless they perform some post-processing, such as NSG. MIRAGE addresses global navigability and local precision by incrementally building upper layers with a subset of points that contain long-range links that enable rapid traversal across the graph. As the hierarchy levels increase, the graphs get sparser. This hierarchical structure allows searches to begin in the upper layers for efficient broad exploration, and progressively narrow down to precise results in the bottom layer, ensuring good search performance.

Consequently, MIRAGE builds a hierarchical structure, where the bottom layer (Layer 0) of the graph is constructed using a refinement-based approach, and then creates layers on top (Layers 1 to N) following an increment-based approach. During search, MIRAGE uses the greedy beam search, starting from the top layer and moving down until the bottom layer is reached where the k nearest neighbors to the query point are found. New inserts to the constructed structure occur in the same manner as increment-based construction, going through ANNS to find their correct neighbors.

3.2 Graph Construction

The construction of MIRAGE is given in Algorithm 2 and follows two steps: Layer 0 construction (lines 1-10 in Algorithm 2) and the construction of Layers 1 to N (lines 12-28 in Algorithm 2).

It constructs the bottom layer (Layer 0) of the graph using a refinement-based approach. To be exact, it initializes a graph connected to S random neighbors and then runs Algorithm 3, which finds neighbors using a traditional NN-Descent algorithm and prunes edges at the same time using the RNG rule. As opposed to a traditional refinement-based approach that automatically forms edges between neighbors of neighbors (i.e., two-hop neighbor edges), we use insights from RNNDescent to first check whether the RNG rule is satisfied before adding the edge (lines 11-13 of Algorithm 3). This speeds up construction time by eliminating unnecessary addition of edges that would be pruned later anyway. We maintain a flag for each vertex indicating whether it is "new" or "old"

Algorithm 2 MIRAGE Construction**Require:** Dataset $P, S, M, C, Iter, R$ level normalization $m_L \in (0, 1)$ **Ensure:** Hierarchical graph structure $G = (L_0, L_1, \dots, L_\ell)$

```

1: Initialize Layer 0:  $L_0 \leftarrow \text{RandomGraph}(S)$ 
2: Set all flags in  $L_0$  to "new"
3: for  $r \leftarrow 1$  to  $R$  do
4:   for  $iter \leftarrow 1$  to  $iter$  do
5:      $\text{UPDATE}(L_0, M)$ 
6:   end for
7:   if  $r \neq R$  then
8:      $\text{ADDREVERSEEDGES}(L_0)$ 
9:   end if
10: end for
11:  $L_0 \leftarrow$  Constructed bottom layer
12:  $entry\_point \leftarrow$  Choose a random node from  $L_0$ 
13: for each point  $p \in D$  do
14:    $level \leftarrow \text{RandomLevel}(m)$ 
15:   if  $level > 0$  then
16:     for  $l \leftarrow level$  to 1 do
17:        $closest \leftarrow \text{ALGORITHM 1}(p, entry\_point, 1, l)$ 
18:        $entry\_point \leftarrow$  nearest element from  $closest$  to  $p$ 
19:     end for
20:     for  $l \leftarrow 0$  to  $level$  do
21:        $neighbors \leftarrow \text{ALGORITHM 1}(p, entry\_point, C, l)$ 
22:        $selected\_neighbors \leftarrow \text{RNG RULE}(\text{layer } l, M)$ 
23:        $\text{ADDREVERSEEDGES}(\text{layer } l)$ 
24:     end for
25:   end if
26:   if  $level > \text{max level in } G$  then
27:     Update entry point:  $entry\_point \leftarrow p$ 
28:   end if
29: end for
30: return  $G$ 

```

to help eliminate duplicate distance comparisons of the same vertices – a common practice in refinement-based approaches. MIRAGE then follows the standard practice in graph-based indexes and adds reverse edges to complete the bottom layer of the graph (line 8 from Algorithm 2). We limit the maximum out-degree of the constructed graph, pruning every vertex in Layer 0 down to M neighbors using the RNG rule (lines 21 and 22 of Algorithm 3), something that RNNDescent does not do. This helps improve MIRAGE's reachability and search performance. Due to its fast construction time, use of the RNG rule to select edges, and its use of reverse edge addition, the refinement-based constructed graph at Layer 0 is best suited to be incorporated into an increment-based approach for the higher layers.

To generate upper layers, a maximum layer index L is chosen using an exponentially decaying probability distribution, normalized by $m_L = \frac{1}{\ln(M)}$, as is done in HNSW. Each point in the dataset is assigned a layer by the distribution and then first inserted at the corresponding layer, where it finds its M nearest neighbors using Algorithm 1, moves down a layer and repeats the same process, stopping after being inserted at layer 1. While selecting neighbors at each layer, if the value of M is exceeded for any vertex, we use the RNG rule to prune edges. We also add reverse

edges to the graph to improve navigability. Using ANNS in the construction of the upper layers allows us to get the best of both worlds in terms of refinement and increment-based construction. Since bottom layer is constructed using refinement-based approach over the entire dataset, and the hierarchy constructions uses ANNS on only a subset of the dataset, we get the improved speed of the refinement-based approach, with the improved connectivity of using an increment-based approach for the upper layers. This ensures that, during search, we get optimal entry points at each layer, leading to improved query accuracy and efficiency.

Algorithm 3 UPDATE

Require: Graph $G = (V, E)$, vertex $u \in V$, maximum out-degree M

Ensure: Updated edge set $E \subset V \times V$

```

1: for each  $u \in V$  do
2:    $U \leftarrow \{v \mid (u, v) \in E\}$ 
3:   Sort  $v \in U$  in ascending order of  $\delta(u, v)$ 
4:    $U' \leftarrow \emptyset$ 
5:   for each  $v \in U$  do
6:      $f \leftarrow \text{true}$ 
7:     for each  $w \in U'$  do
8:       if both flags of  $v$  and  $w$  are "old" then
9:         continue
10:      end if
11:      if  $\delta(u, v) \geq \delta(v, w)$  then // RNG rule check
12:         $f \leftarrow \text{false}$ 
13:         $E \leftarrow (E \setminus (u, v)) \cup (w, v)$ 
14:        break
15:      end if
16:    end for
17:    if  $f$  then
18:       $U' \leftarrow U' \cup \{v\}$ 
19:    end if
20:  end for
21:  if  $|U'| > M$  then
22:    RNG RULE( $U'$ ,  $M$ )
23:  end if
24:  Set the flag "old" for all vertices in  $U'$ 
25: end for

```

3.3 Search

MIRAGE's search algorithm follows most graph-based indexes, namely standard greedy search, with the addition of hierarchy. It starts with a pre-defined entry point at the top-most layer of the graph and then performs Algorithm 1 on each layer with $k = 1$, i.e., until it finds a single nearest neighbor to the query on said layer. Then it moves to the same point on the layer below and repeats the process, finally concluding at the lowest layer once the k nearest neighbors to the query are found. Intuitively this process allows for the best entry point at each layer, helping to improve query accuracy and throughput. During search, the value of C is of the utmost importance. A higher C will result in a slower but more accurate search, whereas a lower C will result in a faster but less accurate search.

Algorithm 4 MIRAGE Search

Require: Multi-layer graph G , query element q , number of nearest neighbors to return K , size of the dynamic candidate list C

Ensure: K nearest elements to q

```

1:  $N \leftarrow \emptyset$  ▷ Set for the current nearest elements
2:  $ep \leftarrow$  get entry point for MIRAGE
3:  $L \leftarrow$  level of  $ep$  ▷ Top layer for MIRAGE
4: for  $lc = L$  to 1 do
5:    $W \leftarrow$  Algorithm 1( $q, ep, C = 1, lc$ )
6:    $ep \leftarrow$  get nearest element from  $W$  to  $q$ 
7: end for
8:  $W \leftarrow$  Algorithm 1( $q, ep, C, lc = 0$ )
9: return  $K$  nearest elements from  $N$  to  $q$ 

```

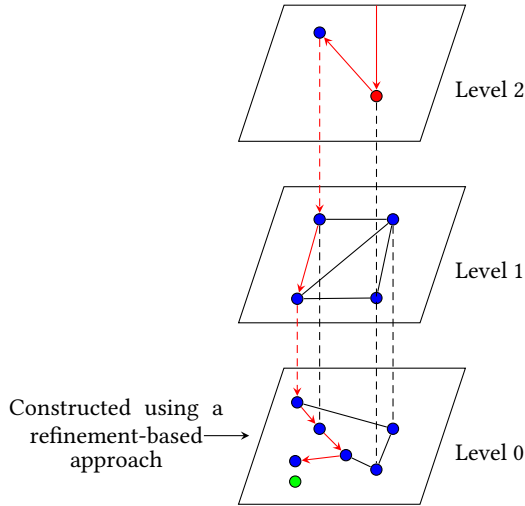


Fig. 4. MIRAGE construction and search. The search path is shown by red arrows with a fixed entry points and the query point is in green. Due to Layer 0 being constructed with a refinement-based approach, MIRAGE is much faster than HNSW in terms of indexing speed. MIRAGE also has a sparser bottom layer which allows for its search efficiency to be better than HNSW, as much as 2x.

3.4 Handling Updates

Vector datasets can often evolve over time, requiring indexing approaches that can efficiently support dynamic updates [43]. These updates include insertions and deletions, though in-memory graph-based methods typically overlook deletions [28]. Consequently, in this paper we focus on handling incremental inserts in MIRAGE and leave incremental deletions for future work. As noted earlier, refinement-based solutions such as NN-Descent, RNNDescend, and NSG do not handle incremental insertions.

For incremental inserts, MIRAGE follows the same method it uses for Layers 1 to N and starts by assigning a layer to the new point. Next it inserts that point at its highest layer, performs Algorithm 1 to find its M nearest neighbors, and moves down a layer to repeat the same process. The difference from construction (Algorithm 2) is that an update does not stop at Layer 1, but proceeds to Layer 0. This is because we want all the points to be present at Layer 0, since that is

where the search concludes. If any point exceeds M connections, RNG rule is used to prune edges. Using an increment-based approach for incremental inserts allows us to preserve the quality of the graph and ensures that we do not have to reconstruct the graph from scratch. To effectively compare the accuracy of insertion-based solutions such as MIRAGE and HNSW, existing literature states that it is sufficient to evaluate their KNN search performances, as the neighbors of each newly inserted point are selected from the approximate nearest neighbors returned by the search [28]. As shown in our experimental results, MIRAGE delivers faster query results with the same precision as HNSW, indicating that MIRAGE can handle insertions efficiently.

3.5 Complexity of Algorithms

The overall construction complexity of MIRAGE is: $O(n^{1.14})$, where n is the number of vertices in the dataset.

The construction of the bottom layer dominates the overall construction time with an empirical complexity of approximately $O(n^{1.14})$. After Layer 0 is constructed, the increment-based algorithm is used to build additional layers hierarchically. The complexity of this construction alone is $O(n \cdot \log(n))$, but since the construction of the bottom layer has a higher complexity, the total construction complexity remains $O(n^{1.14})$.

The search complexity is primarily influenced by the hierarchical nature of the algorithm and is $O(\log(n))$. The search process proceeds layer-by-layer, utilizing a greedy search strategy to progressively narrow down the search space until it reaches the bottom layer. The initial guidance provided by the upper layers significantly localizes the search scope, ensuring that the total search time scales logarithmically with the size of the dataset.

We note that neither HNSW, NN-Descent, nor MIRAGE provide theoretical guarantees on graph connectivity, therefore we rely on empirical results for analysis of our method's effectiveness [37].

4 Evaluation

We evaluate MIRAGE through a series of experiments on real-world datasets.

4.1 Experimental Setup

Our experiments involve comparing MIRAGE with existing methods in terms of construction and search performance, looking at the comparative qualities of the graphs created by different methods, and verifying the effectiveness of our method with ablation studies.

Datasets. We conduct our experiments on 7 real-world datasets with different sizes and dimensions (See Table 2). These datasets are commonly used for the evaluation of different ANNS indexing methods. Gist1M [4] consists of 1 million high-dimensional (960-dimensions) GIST descriptors extracted from images, along with 1,000 query vectors. The SIFT10M dataset [4] consists of a collection of 10M and 100M base vector respectively, and 10K query vectors. All of the vectors are 128-dimensional local SIFT descriptors from INRIA Holidays images. The Paper dataset [52] consists of about 2M base vectors and 10K query vectors. The dataset is generated by extracting and embedding the textual content from an in-house corpus of academic papers. Crawl dataset [3] provides a corpus for collaborative research, analysis and education. The text is converted to 300-dimension vectors with fastText55. Glove (Global Vectors for Word Representation) [40] is a word vector dataset produced by the corresponding Glove unsupervised learning algorithm. This dataset is the pre-trained word vectors from Wikipedia 2014. Tiny image [1] dataset extracts 5 million images from 80 million images of size 32×32 pixels collected from the Internet, crawling the words in WordNet. Deep10M consists of the first million vectors of the Deep1B dataset [56]. The vectors were obtained by running a convolutional neural network on an image collection, reduced to 96 dimensions by principal component analysis and subsequently l2-normalized.

Dataset	Dimension	# Base	# Query	LID	Type
Tiny5M	384	5,000,000	1,000	-	Image
Deep10M	96	10,000,000	10,000	12.1	Image
SIFT10M	128	10,000,000	10,000	9.3	Image
GIST1M	960	1,000,000	1,000	18.9	Image
Crawl	300	1,989,995	10,000	15.7	Text
GloVe	100	1,183,514	10,000	20.0	Text
Paper	200	2,029,997	10,000	-	Text

Table 2. Summary of datasets with dimensions, base size, query size, LID, and data type. LID [25] denotes how many “degrees of freedom” are necessary to describe the dataset and is used to measure the difficulty of answering NN queries over it; higher LID means a harder dataset.

Performance Metrics. To measure each method’s overall performance, we employ metrics related to index construction and search. For index construction, we evaluate the index construction time and peak memory usage. Some index characteristics, such as average out-degree and the number of connected components, are recorded; they indirectly affect index construction efficiency and size. For search, we evaluate search speed and accuracy. Search speed is measured in QPS, which is the ratio of the number of queries ($\#q$) to the search time (t): $\frac{\#q}{t}$. We evaluate search accuracy using two metrics: (1) recall, which measures the ratio of successfully retrieved ground truth k -nearest neighbors to k (see Definition 4 above) and (2) average distance ratio (ADR) (See Definition 5 above) [17, 18, 23, 38, 39, 46], which represents the mean of the distance ratios (equivalent to the average relative error on distance plus one) of the retrieved k objects relative to the ground truth k -nearest neighbors.

Methods and Parameters. We select 6 graph-based indexing methods to compare against MIRAGE, namely HNSW [30], NN-Descent [21], NSG [16], RNNDescent [34], Vamana (DiskANN) [45], and LSH-APG [60]. We selected HNSW and NSG because of their superior search performance in multiple studies and their popularity. HNSW is also the most popular increment-based graph index. NN-Descent is the most popular refinement-based approach and is used as the first step in many graph-based indexes including NSG. The other three: RNNDescent, Vamana, and LSH-APG, are recent proposals optimized for index construction time.

We configure each method’s parameters to maximize its search performance. For the Sift, Gist, Glove, and Crawl datasets we use the parameters provided by [53], which are found by using a grid search within the parameter space. For the Deep dataset we follow [45] where the parameters are found using a parameter sweep. For the Tiny dataset, we follow guidance from [28], and for the Paper dataset we follow the parameters set by the original paper [52].

Platform. Experiments are run on a Linux server of Xeon(R) Platinum 8380 CPU and 1 Terabyte of DDR4 RAM. In our experiments we set the thread count to 128. All benchmarks were implemented in C++. Our source code is available at: <https://github.com/dsg-uwatloo/mirage>.

4.2 Memory Cost

Figure 5 shows the peak memory needed to construct each index, which is different from index size. We choose this metric because a previous study [28] indicates that index size mainly depends on the average out-degree of the graph, and the smaller the value, the smaller the index size [54]. For completeness we show those values and discuss them in Section 4.5. Because of this, methods that prune edges using methods such as the RNG rule have fewer edges and therefore lower index size.

However, this does not tell the whole story – in order to generate graphs using a refinement-based approach, many indexes start with the K-Graph approach, which has a high maximum out-degree during the construction process, incurring high memory cost. This applies to approaches like NSG and NN-Descent [28]. Therefore, following the current literature, we consider memory cost in addition to index size.

As can be seen in Figure 5, refinement-based approaches such as NSG and NN-Descent use the most memory to build the index, because they first build an approximate KNN graph. NSG uses NN-Descent to build a graph before pruning edges, and is therefore bound by its peak memory usage. Even though HNSW contains multiple layers, since it uses an increment-based approach starting with an empty graph, it does not build a K-Graph and therefore has a lower maximum out-degree than refinement-based approaches during construction. Consequently, it has lower memory use during construction. RNNDescent does not build a K-Graph either, but prunes during the process of building the graph (instead of after construction), which leads to its lower memory use compared to other refinement-based approaches.

MIRAGE's memory overhead is dominated by its bottom layer where it also prunes while refining the graph. Upper layers are generated using an increment-based approach and do not affect memory consumption much.

4.3 Index Construction Performance

We evaluate the index construction efficiency of all the methods in terms of construction time. The results are shown in Figure 6. Depending on the dataset, NN-Descent or RNNDescent constructs their indexes fastest, with MIRAGE taking slightly more time than RNNDescent to construct. When the size of the dataset is small, the slowdown is marginal, and even when the dataset size increases, it is only 20% slower at its worst. The slowdown is due to the hierarchical structure of MIRAGE in which Layers 1 to N require computing ANNS to construct. However, since the layers are only constructed on a subset of the total dataset, the slowdown is negligible. Vamana performs well on 2 datasets of 1M vectors, but is slow to construct as the number of vectors increases due to its high out degree and using ANNS to form edges for each point in the dataset. Similarly, HNSW uses ANNS to construct the entire dataset, causing the high construction times compared to the other methods tested. MIRAGE constructs its index faster than HNSW and NSG on all benchmarked datasets – up to 6× faster than HNSW and 7× faster than NSG, depending on the dataset. MIRAGE constructs its index faster than NSG due to the fact NSG must create a KNN graph using NN-Descent first before pruning instead of pruning during the construction process like Layer 0 of MIRAGE. LSH-APG is significantly slower than MIRAGE on all datasets (as much as 6×).

4.4 Search Performance

For each dataset, we plot QPS vs recall to show the search performance of MIRAGE compared to the benchmarked methods (Figures 7, 8, 9, 10). This requires adjusting search parameters for each method, namely the value of C in Algorithm 1. We search the graphs with different values of C starting at 1, going until 1024. Generally, the higher the value of C , the higher the recall and the lower the QPS. A higher QPS generally shows higher search efficiency and is related to the number of distance computations that a search performs [53]. We look at different k values for each dataset and method, namely $k = 1$, $k = 10$, $k = 20$, and $k = 100$. We plot the recall on each dataset from 0.7 to 1; including full range does not provide more information and makes it hard to read.

NN-Descent does not have competitive recall performance as compared to the other solutions on many datasets, due to the issues we mentioned above with K-Graph. Both LSH-APG and Vamana have competitive recall performance but are orders of magnitude worse than the top performing solutions in terms of QPS, similar to the findings in previous work [57]. RNNDescent performs

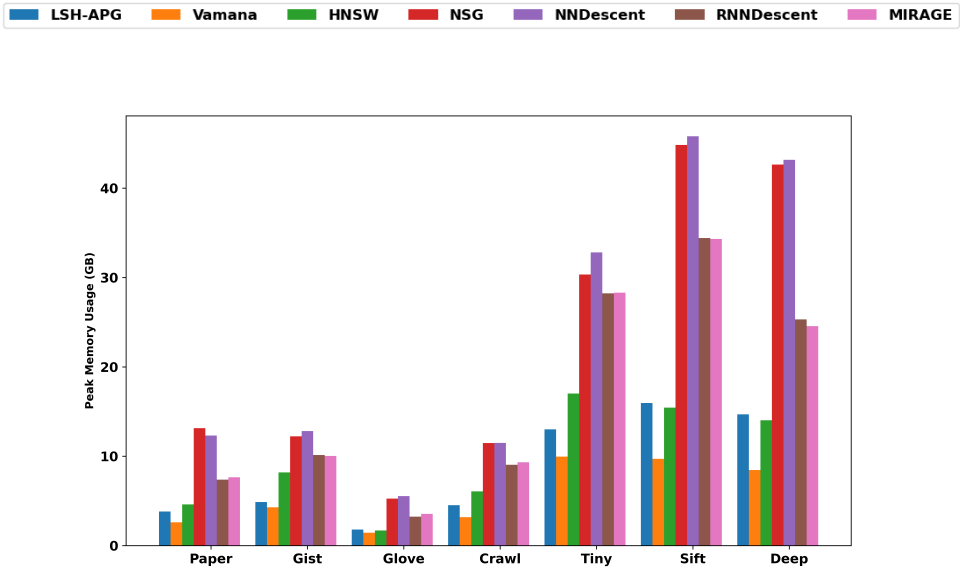


Fig. 5. Comparison of peak memory usage for different methods

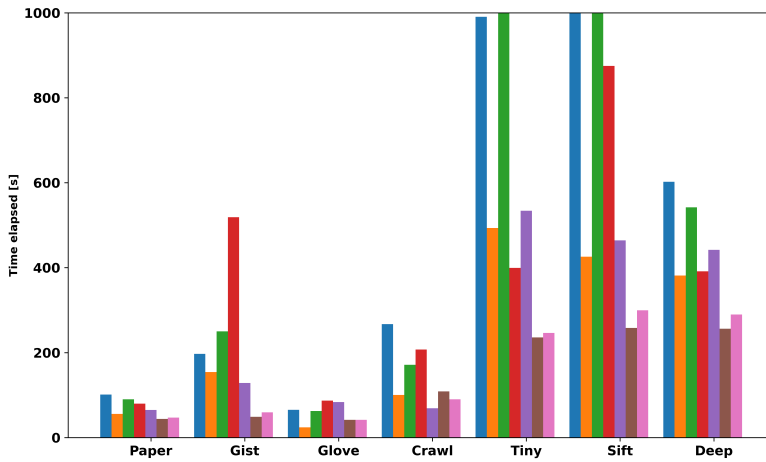


Fig. 6. Indexing time of different graph-based methods. We cap the y-axis at 1000s for readability reasons. For the Tiny5M dataset, HNSW takes 1856.485s and for Sift10M HNSW takes 1051.475s and LSH-APG takes 1249.135s

worse as the value of k increases; the original paper includes search performance only at $k = 1$ [34]. Previous work has noted that the performance of graph-based ANNS algorithms can vary wildly depending on the dataset characteristics [53]. Irrespective, on all tested datasets, MIRAGE shows the highest QPS at the highest recalls, performing up to $2\times$ better than HNSW and NSG at the highest recalls.

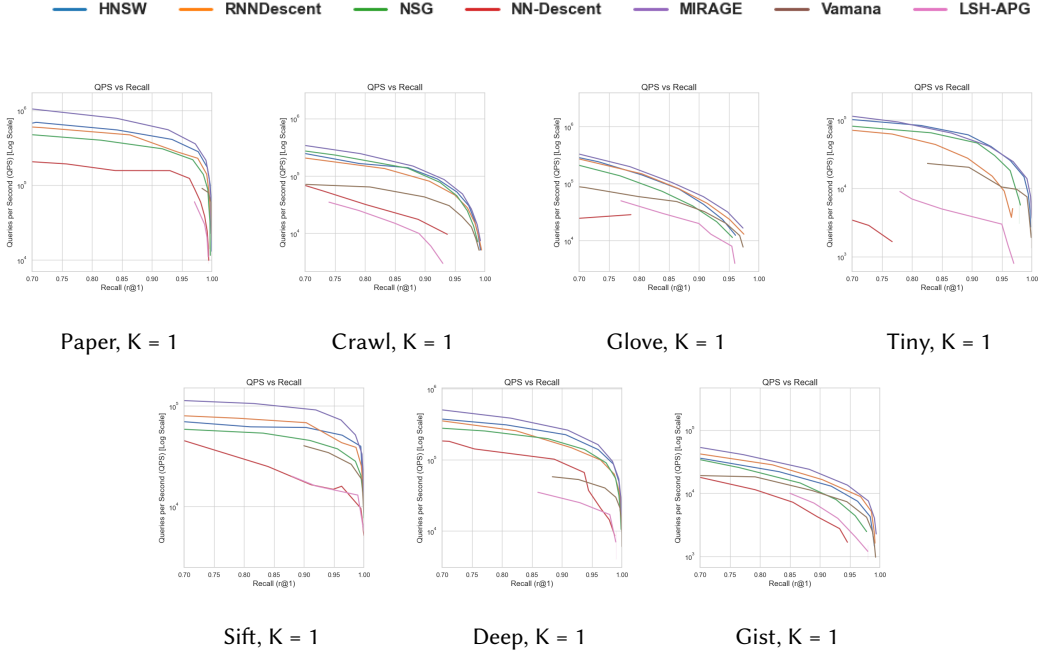


Fig. 7. QPS (log scale) vs Recall of all benchmarked methods and datasets where $k = 1$

In order to have a more fine grained look at search accuracy, we take the top three performing algorithms in terms of search (MIRAGE, NSG, and HNSW) and plot QPS vs ADR for two datasets with high LIDs (Glove and Crawl) and all the k values mentioned above. These are shown in Figure 11. At all k values on both datasets, MIRAGE has the highest QPS while also having the lowest ADR, confirming it is the best solution both in terms of search accuracy and throughput.

Although MIRAGE constructs its index faster than HNSW and NSG, it is still able to have better search efficiency than both solutions at the same or better accuracy. The following sections discuss experiments that explain the reasons for this.

4.5 Graph Qualities

As mentioned in Section 4.2, the index size is mostly a function of the average out-degree of the graph [53]. Thus we consider average out-degree and index size for each graph generated from all 7 datasets (Table 3). NN-Descend produces an index with the highest average out-degree out of all methods considered, and therefore has the largest index size. This is because it does not prune edges. Vamana constructs a graph with the second highest average out-degree on all datasets. Depending on the dataset, HNSW (Layer 0) or LSH-APG have the third highest average out-degree, and RNNDescend or NSG have the fourth-highest average out-degree. MIRAGE has a lower average out-degree than RNNDescend due to its maximum out-degree restriction. However, MIRAGE has to store the upper layers of the graph. Since the layers only contain a subset of the total dataset, this is not much extra overhead, around 0.5GB extra on the Tiny dataset. In terms of search performance, the search cost of the greedy search algorithm is the product of the length of the search path and the out-degree of each point in the path [54]. The low average out-degree of MIRAGE allows it to

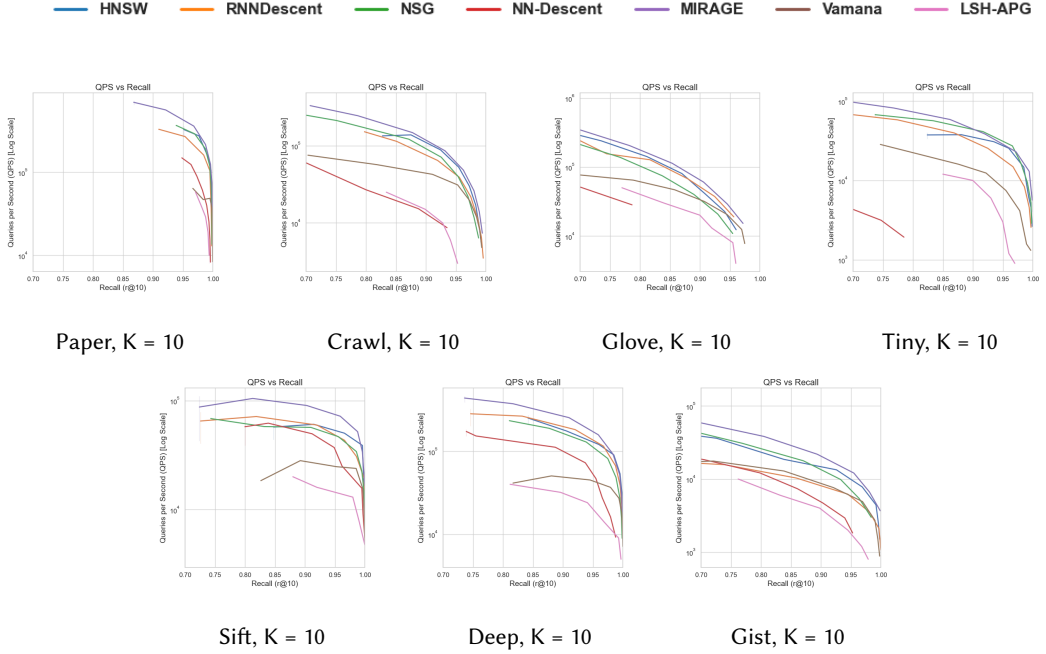


Fig. 8. QPS (log scale) vs Recall of all benchmarked methods and datasets where $k = 10$.

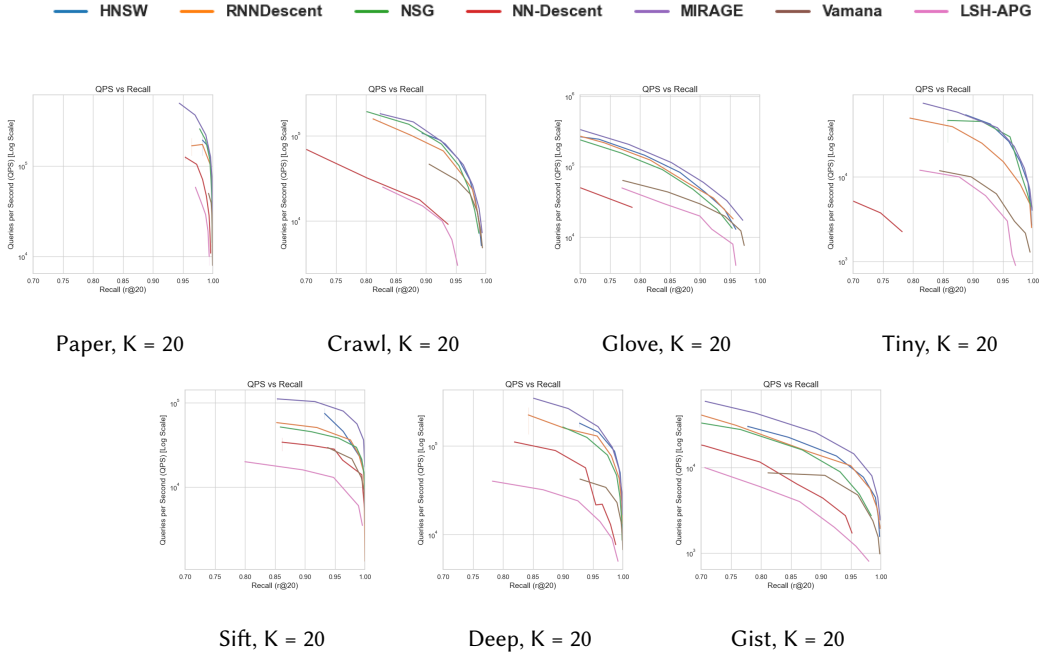
perform fewer distance computations compared to other solutions, helping it maintain its superior search performance across datasets. We expand on this more in Section 4.8.

4.6 Parameter Experiments

We study the effects of the construction parameters on both construction and search performance. For construction we look at the construction time and for search we look at the QPS vs ADR. All parameter experiments are run on the Gist dataset, and we set $k = 100$ for all search experiments. We choose $k = 100$, since when running initial experiments we observed recall degradation at this k value, prompting us to test the impact of graph density/construction parameters on search performance at $k = 100$ value. We choose ADR to get a more fine-grained look at search accuracy performance.

First we look at S , which represents the initial out-degree of the randomly connected graph at Layer 0 before refinement. As Figure 12 depicts, increasing S causes construction time to increase. This is due to a denser graph being constructed, with each vertex having more neighbors. We see a similar phenomenon in search, where there is a direct relationship between the graph density and the QPS vs ADR trade off. A denser graph (higher value of S) has lower QPS and has less accuracy degradation. The decrease in QPS is due to more distance computations being done, related to the average out-degree of a denser graph. Empirically we find the best search performance in terms of QPS and accuracy when $S = 32$ on most datasets tested; even when $S = 60$, MIRAGE is constructed $2.6\times$ faster than HNSW and $5\times$ faster than NSG.

Next we look at R , which represents the number of iterations of reverse edge additions. We fix the value of $Iter$, the number of refinement iterations to 15. The results, shown in Figure 13, reveal

Fig. 9. QPS (log scale) vs Recall of all benchmarked methods and datasets where $k = 20$

Graph	Gist		Glove		Crawl		Paper		Sift		Tiny		Deep	
	AOD	IS	AOD	IS	AOD	IS	AOD	IS	AOD	IS	AOD	IS	AOD	IS
HNSW Layer 0	21.8	3.7	15.9	0.5	13.8	2.3	26.8	1.7	43.7	6.4	26.9	7.7	46.5	5.3
MIRAGE Layer 0	17.4	3.6	15.6	0.5	10.1	2.3	11.0	1.6	24.9	5.7	21.3	7.5	25.1	4.5
NSG	12.5	3.6	9.6	0.5	10.3	2.3	18.6	1.7	28.0	5.8	18.2	7.5	32.8	4.8
RNNDescent	22.1	3.7	20.7	0.5	11.7	2.3	13.0	1.6	26.8	5.8	32.8	7.8	27.5	4.6
NN-Descent	100.0	3.9	100.0	0.9	80.0	2.8	100.0	2.3	90.0	8.1	100.0	9.0	90.0	6.9
Vamana	67.2	3.8	56.4	0.7	39.8	2.6	66.2	2.1	69.4	7.5	69.3	8.6	69.3	6.4
LSH-APG	32.4	3.8	29.6	0.6	31.1	2.5	35.3	1.8	36.6	6.3	33.0	8.0	35.7	5.0

Table 3. Average vertex out-degree (AOD) and Index size (IS) for each method. Index size is represented in GB.

that an increased R value improves search performance and also increases construction time. This

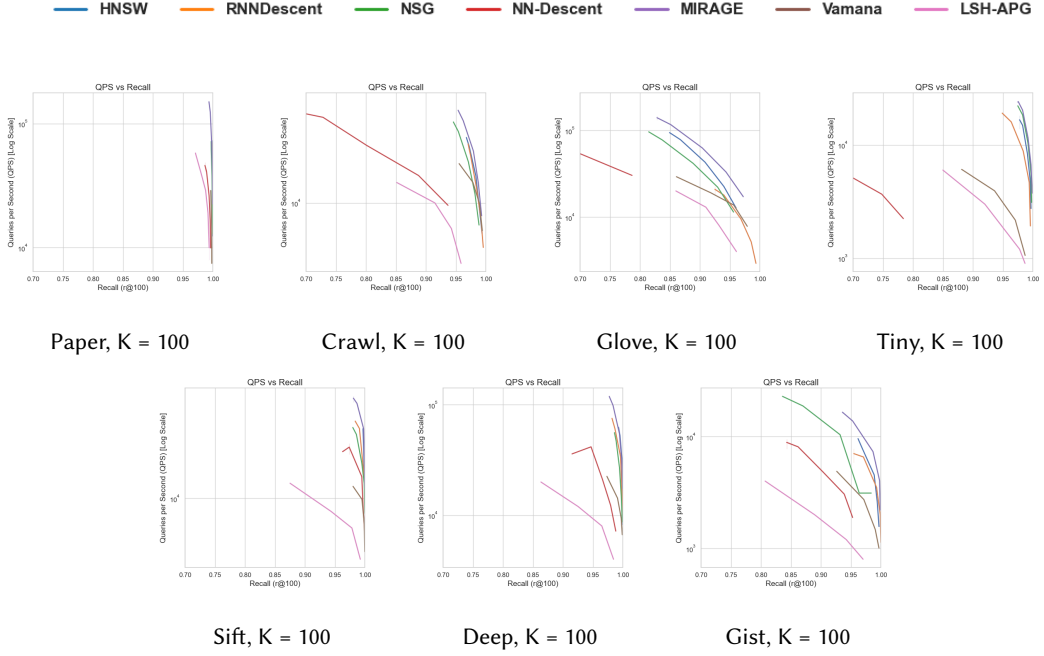


Fig. 10. QPS (log scale) vs Recall of all benchmarked methods and datasets where $k = 100$.

illustrates the importance of reverse edge addition to the Layer 0 of MIRAGE for optimal search performance.

Lastly, we look at *Iter*, which represents the number of refinement iterations MIRAGE goes through at Layer 0. We fix $R = 4$ in these experiments. The results in Figure 14 show there is a negligible difference in construction time between all values, with higher values of *Iter* yielding marginally higher construction times. The main insight here is that after a certain amount of refinement iterations, the improvement in search becomes negligible and can even make the search perform worse due to insufficient opportunities to add reverse edges at a fixed R value. Due to this, we find a less drastic difference between R and *Iter* to have the best trade off between construction and search performance, for example $R = 5$ and *Iter* = 12 instead of $R = 4$ and *Iter* = 50.

Overall, we find that having a denser graph at Layer 0 can help reduce search accuracy degradation at lower recalls, with the trade-off of reduced QPS. Depending on the application needs in terms of search accuracy and throughput, MIRAGE can be configured for best search performance in both scenarios.

4.7 Ablation Study

We investigate the hierarchical search component of MIRAGE in ablation studies to determine its effectiveness. To test the benefits of the hierarchy, we construct MIRAGE with and without hierarchy, and search both structures. We keep the average out-degree of Layer 0 of hierarchical MIRAGE the same as the version without hierarchy for a fair comparison. We note that this is the worst-case scenario for MIRAGE since its benefits come from reducing the density of its bottom layer and getting assistance from the hierarchy to navigate to a good entry point and quickly search the lower layer.

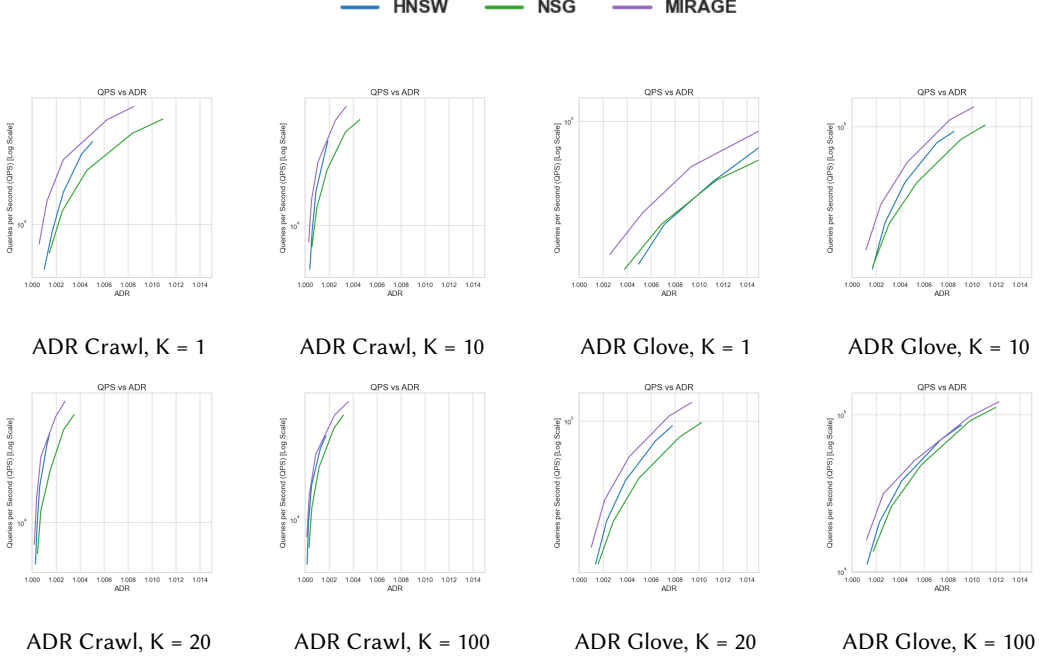


Fig. 11. QPS (log scale) vs ADR of the Glove and Crawl datasets where $k = 1$, $k = 10$, $k = 20$, and $k = 100$.

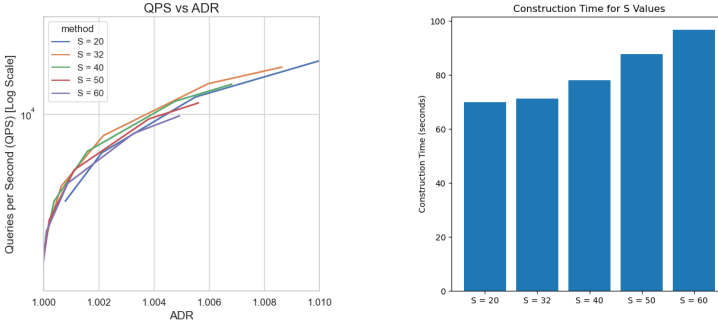


Fig. 12. Effect of different S values on construction time and search performance (QPS vs ADR)

For searching, we find random entry points in the non-hierarchical graph, as they do not have entry points from upper layers, and for the hierarchical version of MIRAGE we use our standard search algorithm. Figure 15 shows the results on the Gist and Paper datasets. The results show that even when the out-degree is the same, the hierarchical solution is able to achieve better search performance due to providing better entry points to the bottom layer than randomly choosing a start point. This is due to the fact that the upper layers are constructed using an increment-based approach, creating a higher quality graph.

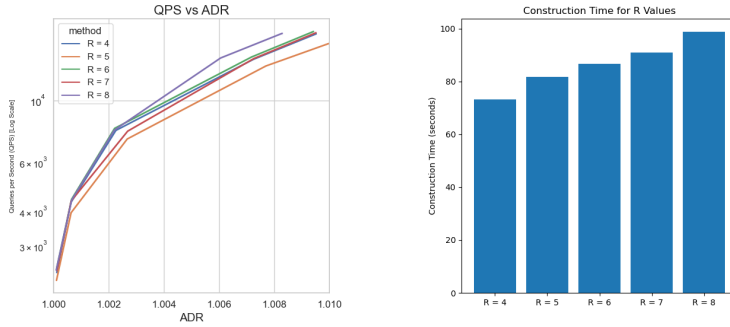


Fig. 13. Effect of different R values on construction time and search performance (QPS vs ADR)

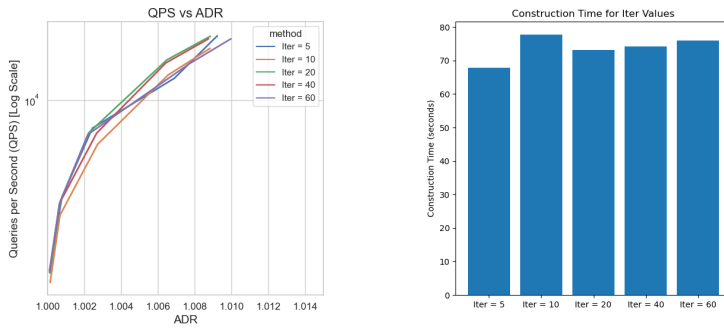


Fig. 14. Effect of different Iter values on construction time and search performance (QPS vs ADR)

Table 4. Number of distance computations to achieve 0.99 recall with $k = 100$. Number in parenthesis represents the difference between HNSW and MIRAGE.

	HNSW	MIRAGE
Paper	3936 (2.4× greater)	1588
Crawl	4319 (2.4× greater)	1758
Gist	25161 (1.7× greater)	14776
Sift	1474 (2× greater)	716

4.8 Discussion

In this section we discuss the reasons why MIRAGE performs better than other methods in terms of search performance; we already discussed why it performs well in terms of index construction time in Sections 3.2 and 4.3. Recall that the search cost of the greedy search algorithm is the product of the length of the search path and the out-degree of each point in the path [54]. The out-degree of the points determines the number of distance computations that a method needs to perform.

Table 4 shows the number of distance computations needed to achieve 0.99 recall when $k = 100$ for MIRAGE and HNSW – a meaningful comparison since they both use a hierarchical greedy search. The results show that MIRAGE performs around 2× fewer distance comparisons than HNSW during the search process. This is what leads to MIRAGE’s superior QPS. The reason for the fewer

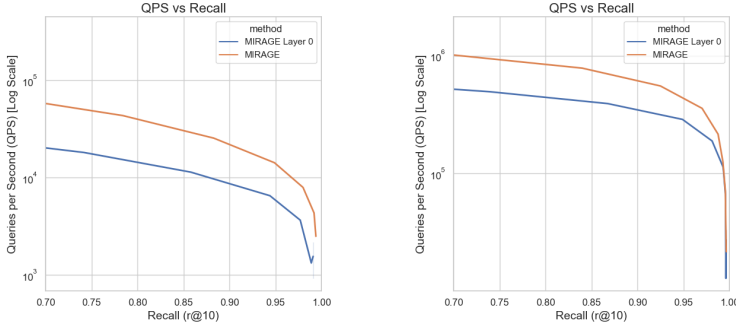


Fig. 15. Ablation Study on MIRAGE Layer 0 vs Hierarchical MIRAGE . Gist dataset is shown on the left, Paper dataset is shown on the right

computations is because MIRAGE has a lower average out-degree than HNSW at Layer 0. In addition, maintaining a sparse lower layer reduces the chances of MIRAGE getting caught in local optima, since too many local connections in a graph lead to more local optima, which may hinder the search accuracy [54].

As shown in the ablation study, adding hierarchy improves the search performance of MIRAGE by having longer edges that can traverse the whole graph quickly and provide good entry points for the lower layer. There is existing work suggesting that HNSW does not benefit that much from hierarchy due to a small subset of well-connected nodes (hubs) naturally forming a “highway” structure at Layer 0 [27, 33]. We find in our ablation study that hierarchy helps MIRAGE likely due to the fact that Layer 0 and upper layers are constructed using different approaches – Layer 0 with a refinement-based approach and upper layers with a increment-based approach. This is different from HNSW, where all layers are constructed using the increment-based algorithm. MIRAGE benefits from having this “highway” structure in its upper layers in a way that HNSW perhaps does not.

5 Related Work

We discussed background related to vector indexing and ANNS in Sections 2.2 and 2.3. In this section we focus on recent works that focus on improving the performance of these methods.

Vamana [45] is the graph-based algorithm that serves as the foundation for the DiskANN system. The construction of a Vamana graph begins with an initialization phase where each vertex is assigned a set of random out-neighbors. Then, the algorithm iterates through all points in a random order, running a greedy search to identify candidate neighbors and applying a robust pruning mechanism to refine the edges. A key feature of Vamana is its tunable α parameter, which controls the trade-off between graph connectivity and search efficiency by enforcing that each step in a search path results in a multiplicative distance reduction. The algorithm makes two passes over the data, first with $\alpha = 1$ and then with a user-defined α greater than 1, ensuring that long-range connections are established for improved search performance. The search process in Vamana follows a best-first traversal strategy where it starts at a designated entry point (usually the dataset medoid) and greedily explores nodes, guided by distance comparisons. The key advantage of Vamana lies in its ability to construct graphs with fewer search hops compared to other methods, making it well-suited for both in-memory and SSD-based indices.

The Hierarchical Voronoi Structure (HVS) [28] is a graph that begins by partitioning the space into hierarchical Voronoi cells, where each data point belongs to exactly one cell. This results in a

hierarchy of Voronoi diagrams, with seed points from higher layers appearing in lower layers. In each layer, a graph is constructed where nodes correspond to Voronoi cells, and edges are formed based on proximity between cells. The edge selection strategy follows principles similar to HNSW. To improve efficiency, a density-based allocation strategy is used, ensuring that fewer data points are included in deeper layers. The search process in HVS follows a top-down hierarchical traversal approach combined with a base-layer graph search. First, a multi-level distance book is computed to store distances between the query and Voronoi cell representatives at different levels. The search begins at the coarsest level, identifying the nearest Voronoi cell as the global entry point. It then progresses layer by layer, where each visited cell's representative determines the entry point for the next lower layer. If a locally nearest Voronoi cell is found before reaching the base layer, the search can terminate early, optimizing efficiency.

LSH-APG (Locality-Sensitive Hashing Approximate Proximity Graph) [60] is an approach that combines locality-sensitive hashing (LSH) with graph-based search methods. LSH-APG constructs a proximity graph using an increment-based approach. Instead of relying solely on traditional graph-based methods, LSH-APG uses LSH to preselect entry points for each new insertion. Additionally, an LSH-based pruning condition is used to filter out distant neighbors during insertion, further reducing unnecessary computations. The search process in LSH-APG starts by using the LSH index to quickly retrieve high-quality entry points in the graph. Once a good starting point is identified, a greedy graph traversal is performed, where the query moves iteratively towards closer neighbors in the APG.

In both years of the SISAP Indexing challenge [47, 48] participants compete on construction time and search performance of the indexing methods they submit. One method that stands out in these competitions is The Continuous Refining Exploration Graph (crEG) [22], which introduces a dynamic pruning strategy based on the average neighbor distance, continuously refining edge quality during and after construction. Similarly, a new framework [57] for proximity graph construction introduces the alpha-pruning strategy, which incorporates angular constraints to improve neighbor selection and graph sparsity. Both methods aim to improve upon pruning using only the RNG rule. MIRAGE uses the RNG rule for pruning, making it easier to be adapted into existing libraries, but could also be adjusted to different methods of pruning.

In addition to algorithmic advancements, there are solutions that aim to improve graph-based ANNS performance by using hardware accelerators. SONG [59] is a GPU-based ANNS implementation that offers a significant speedup over CPU counterparts, outperforming FAISS by leveraging GPU-centric strategies. GANNS [58] builds on this by enhancing parallelism and using an NSW-based proximity graph. More recently, CAGRA [35] has utilized modern GPU techniques, such as warp splitting, to achieve substantial performance improvements over other GPU-based methods. However, these graph-based approaches, including SONG, GANNS, and CAGRA, face limitations when scaling to larger datasets, as they require the entire graph index to reside in GPU memory.

Beyond GPUs, there is use of other accelerators like FPGAs for ANNS. FPGA-based methods have been investigated, e.g., vStore [26] and TPU-KNN [13].

Various systems and algorithms are also being developed for *filtered queries*. These queries combine ANNS over vector space with predicates over structured data, and are crucial for applications that involve multiple data modalities, such as finding similar images while applying metadata-based filters. Examples of graph-based solutions include Filtered-DiskANN [20], Milvus [51], ACORN [37], and iRangeGraph [55]. Efficiently executing filtered queries remains a challenge, requiring both fast query performance and query semantics to handle a diverse range of predicates that may not be predefined.

Pre-filtering and post-filtering are the main strategies for filtered queries. Pre-filtering first applies the structured predicate and then performs ANNS on the vector space. It becomes inefficient

as selectivity increases or when dealing with large datasets. Post-filtering performs the ANNS first, then applies the filter, but often requires expanding the search scope, leading to inefficiencies for low-selectivity predicates.

6 Conclusion

In this work, we present MIRAGE, a novel graph-based multilayered index for Approximate Nearest Neighbor Search (ANNS) that combines the benefits of both refinement-based and increment-based approaches. Using a novel construction strategy that builds the bottom layer of the graph index using a refinement-based approach and higher layers using an increment-based approach, we ensure fast construction times while maintaining high search accuracy and supporting incremental inserts. Our extensive evaluation on multiple real-world datasets demonstrates that MIRAGE consistently outperforms state-of-the-art increment-based methods such as HNSW in terms of index construction and query recall, query speed, and refinement-based methods such as NSG, NN-Descent, and RNNDescent in terms of query speed and ability to perform incremental inserts without requiring full graph reconstruction. Through studying graph properties and with ablation studies, we show that each component of MIRAGE is providing a benefit, and demonstrate why MIRAGE performs so well across a range of datasets. These results indicate that MIRAGE is well-suited for large-scale ANNS tasks across a variety of applications.

We are following a number of directions for improving MIRAGE and using it in downstream tasks. One immediate issue is to incorporate the ability to handle vector deletions through incremental updates of the MIRAGE index. Vertex deletions in directed graphs is a difficult problem as the vertex is connected to other vertices and many edges need to be dropped [49]. Consequently, most in-memory graph-based indexes ignore deletions [28]. However, there are ways to incorporate them, perhaps by the use of auxiliary data structures [60] and this is an issue under consideration. We believe that there are ways to further improve the performance of MIRAGE by the using hardware accelerators similar to the other graph-based methods mentioned [35, 58, 59]. Lastly, given the superior performance of MIRAGE in both index construction and search, we are interested in using it as the basis of filtered query processing techniques mentioned above.

Acknowledgments

Research supported by Natural Sciences and Engineering Research Council (NSERC) of Canada under Discovery Grant RGPIN-2024-03993.

References

- [1] [n. d.]. <https://www.cse.cuhk.edu.hk/systems/hash/gqr/datasets.html>. [Accessed 08-10-2024].
- [2] [n. d.]. ANNOY library. <https://github.com/spotify/annoy>. Accessed: 08-10-2024.
- [3] [n. d.]. Common Crawl - Open Repository of Web Crawl Data — commoncrawl.org. <https://commoncrawl.org/>. [Accessed 08-10-2024].
- [4] [n. d.]. Evaluation of Approximate nearest neighbors: large datasets — corpus-texmex.irisa.fr. <http://corpus-texmex.irisa.fr/>. [Accessed 08-10-2024].
- [5] [n. d.]. GPT-4. <https://openai.com/research/gpt-4>
- [6] [n. d.]. Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>
- [7] [n. d.]. The vector database to build knowledgeable AI | Pinecone — pinecone.io. <https://www.pinecone.io/>. [Accessed 14-10-2024].
- [8] [n. d.]. vespa.ai. <https://vespa.ai/>. [Accessed 14-10-2024].
- [9] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1, Article 43 (2025), 31 pages. doi:10.1145/3709693
- [10] Brankica Bratić, Michael E. Houle, Vladimir Kurbalija, Vincent Oria, and Miloš Radovanović. 2018. NN-Descent on High-Dimensional Data. In *Proc. 8th Intl. Conf. on Web Intelligence, Mining and Semantics*. Association for Computing Machinery, Article 20, 8 pages. doi:10.1145/3227609.3227643

- [11] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. SPTAG: A library for fast approximate nearest neighbor search. <https://github.com/Microsoft/SPTAG>.
- [12] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5199–5212. https://proceedings.neurips.cc/paper_files/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf
- [13] Felix Chern, Blake Hechtman, Andy Davis, Ruiqi Guo, David Majnemer, and Sanjiv Kumar. 2024. TPU-KNN: K nearest neighbor search at peak FLOP/s. In *Proc. 36th Intl. Conf. on Neural Information Processing Systems (NIPS '22)*. Curran Associates Inc., Article 1127, 13 pages.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. <https://github.com/facebookresearch/faiss>. arXiv:2401.08281 [cs.LG]
- [15] Cong Fu and Deng Cai. 2016. EFANNA : An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph. arXiv 1609.07228. <https://api.semanticscholar.org/CorpusID:9041621>
- [16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754
- [17] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12)*. Association for Computing Machinery, 541–552. doi:10.1145/2213836.2213898
- [18] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. doi:10.1145/3589282
- [19] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proc. 25th Intl. Conf. on Very Large Data Bases. VLDB Endowment*, 518–529.
- [20] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proc. ACM Web Conf. 2023*. Association for Computing Machinery, 3406–3416. doi:10.1145/3543507.3583552
- [21] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast Approximate Nearest-Neighbor Search with k-Nearest Neighbor Graph. In *Proc. 22nd Intl. Joint Conf. on Artificial Intelligence*. 1312–1317. doi:10.5591/978-1-57735-516-8/IJCAI11-222
- [22] Nico Hezel, Kai Uwe Barthel, Konstantin Schall, and Klaus Jung. 2024. An Exploration Graph with Continuous Refinement for Efficient Multimedia Retrieval. In *Proc. of 2024 International Conference on Multimedia Retrieval*. Association for Computing Machinery, 657–665. doi:10.1145/3652583.3658117
- [23] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proc. VLDB Endow.* 9, 1 (Sept. 2015), 1–12. doi:10.14778/2850469.2850470
- [24] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proc. Thirtieth Annual ACM Symposium on Theory of Computing*. Association for Computing Machinery, 604–613. doi:10.1145/276698.276876
- [25] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement. *IEEE Trans. on Knowledge and Data Eng.* 32, 8 (2020), 1475–1488. doi:10.1109/TKDE.2019.2909204
- [26] Shengwen Liang, Ying Wang, Ziming Yuan, Cheng Liu, Huawei Li, and Xiaowei Li. 2022. VStore: in-storage graph based vector search accelerator. In *Proc. 59th ACM/IEEE Design Automation Conf.* Association for Computing Machinery, 997–1002. doi:10.1145/3489517.3530560
- [27] Peng-Cheng Lin and Wan-Lei Zhao. 2019. Graph based Nearest Neighbor Search: Promises and Failures. <https://arxiv.org/abs/1904.02077>. arXiv:1904.02077 [cs.IR]
- [28] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proc. VLDB Endow.* 15, 2 (2021), 246–258. doi:10.14778/3489496.3489506
- [29] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* 45 (2014), 61–68. <https://api.semanticscholar.org/CorpusID:9896397>
- [30] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473

- [31] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. 2018. A Survey of Product Quantization. *ITE Trans. on Media Technology and Applications* 6, 1 (2018), 2–10. doi:10.3169/mta.6.2
- [32] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proc. ACM Manag. Data* 1, 2, Article 197 (June 2023), 25 pages. doi:10.1145/3589777
- [33] Blaise Munyampirwa, Vihan Lakshman, and Benjamin Coleman. 2024. Down with the Hierarchy: The 'H' in HNSW Stands for "Hubs". <https://arxiv.org/abs/2412.01940>. arXiv:2412.01940 [cs.LG]
- [34] Naoki Ono and Yusuke Matsui. 2023. Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search. In *Proc. 31st ACM Intl. Conf. on Multimedia*. Association for Computing Machinery, 1659–1667. doi:10.1145/3581783.3612290
- [35] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *Proc. 40th IEEE Intl. Conf. on Data Engineering*. IEEE Computer Society, 4236–4247. doi:10.1109/ICDE60146.2024.00323
- [36] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Companion of the 2024 International Conference on Management of Data*. Association for Computing Machinery, 597–604. doi:10.1145/3626246.3654691
- [37] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3, Article 120 (May 2024), 27 pages. doi:10.1145/3654923
- [38] Marco Patella and Paolo Ciaccia. 2008. The Many Facets of Approximate Similarity Search. In *Proc. First International Workshop on Similarity Search and Applications*. 10–21. doi:10.1109/SISAP.2008.18
- [39] Marco Patella and Paolo Ciaccia. 2009. Approximate similarity search: A multi-faceted problem. *J. of Discrete Algorithms* 7, 1 (March 2009), 36–48. doi:10.1016/j.jda.2008.09.014
- [40] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proc. 2014 Conf. on Empirical Methods in Natural Language Processing*. Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). Association for Computational Linguistics, 1532–1543. doi:10.3115/v1/D14-1162
- [41] William Pugh. 1990. Skip lists: a probabilistic alternative to balanced trees. *Commun. ACM* 33, 6 (June 1990), 668–676. doi:10.1145/78973.78977
- [42] M M Mahabubur Rahman and Jelena Tešić. 2024. Stratified Graph Indexing for efficient search in deep descriptor databases. *Int. J. Multimed. Inf. Retr.* 13, 3 (2024).
- [43] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. <https://arxiv.org/abs/2105.09613>. arXiv:2105.09613 [cs.IR]
- [44] Martin Skrodzki. 2019. The k-d tree data structure and a proof for neighborhood computation in expected logarithmic time. <https://arxiv.org/abs/1903.04936>. arXiv:1903.04936 [cs.DS]
- [45] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Proc. of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Article 1233, 11 pages.
- [46] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *Proc. VLDB Endow* 8, 1 (Sept. 2014), 1–12. doi:10.14778/2735461.2735462
- [47] Eric S. Tellez, Martin Aumüller, and Edgar Chavez. 2023. Overview of the SISAP 2023 Indexing Challenge. In *Similarity Search and Applications: 16th International Conference*. Springer-Verlag, 255–264. doi:10.1007/978-3-031-46994-7_21
- [48] Eric S. Tellez, Martin Aumüller, and Vladimir Mic. 2024. Overview of the SISAP 2024 Indexing Challenge. In *Similarity Search and Applications: 17th International Conference*. Springer-Verlag, 255–265. doi:10.1007/978-3-031-75823-2_21
- [49] Yao Tian, Ziyang Yue, Ruiyuan Zhang, Xi Zhao, Bolong Zheng, and Xiaofang Zhou. 2023. Approximate Nearest Neighbor Search in High Dimensional Vector Databases: Current Research and Future Directions. *IEEE Data Eng. Bull.* 46, 3 (2023), 39–54. <http://sites.computer.org/debull/A23sept/p39.pdf>
- [50] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268. doi:10.1016/0031-3203(80)90066-7
- [51] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proc. 2021 Intl. Conf. on Management of Data (SIGMOD '21)*. Association for Computing Machinery, 2614–2627. doi:10.1145/3448016.3457550
- [52] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jionggang Ni. 2022. Navigable Proximity Graph-Driven Native Hybrid Queries with Structured and Unstructured Constraints. <https://arxiv.org/abs/2203.13601>.

- arXiv:2203.13601 [cs.DB]
- [53] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978. doi:10.14778/3476249.3476255
 - [54] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47, 3 (2023), 3–21. <http://sites.computer.org/debull/A23sept/issue1.htm>
 - [55] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improving Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6, Article 239 (Dec. 2024), 26 pages. doi:10.1145/3698814
 - [56] Artem Babenko Yandex and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proc. 2016 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 2055–2063. doi:10.1109/CVPR.2016.226
 - [57] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2024. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. arXiv:2410.01231 [cs.DB] <https://arxiv.org/abs/2410.01231>
 - [58] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated Proximity Graph Approximate Nearest Neighbor Search and Construction. In *Proc. 38th IEEE Intl. Conf. on Data Engineering*. 552–564. doi:10.1109/icde53745.2022.00046
 - [59] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *Proc. IEEE 36th Intl. Conf. on Data Engineering*. 1033–1044. doi:10.1109/ICDE48307.2020.00094
 - [60] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (April 2023), 1979–1991. doi:10.14778/3594512.3594527

Received October 2024; revised January 2025; accepted February 2025